

metadamage(?) formats

tsk

March 31, 2026

This document describe some of the internal formats used by the metadamage software. These are at the current time.

- .bdamage.gz
- .lca
- .stat
- .dfit.txt.gz
- .agggregate.stat.txt.gz
- .rlens.gz

1 bdamage format

bdamage files are files that contain counts of mismatches conditional on strand and cycle (position within read). These are generated with metadamage lca or metadamage getdamage. The first 8 bytes magic number determines which bdamage version. If no magic number is present then version0 is assumed.

1.1 version 0

First version of the bdamage file is a single bgzf compressed file. MAXLENGTH occurs once in the beginning of the file, followed by successive blocks of data[1-8]. Block[3-5] indicates the actual mismatch counts for the forward which we have MAXLENGTH times. Block[6-8] indicates the actual mismatch counts for the reverse strand which will also occur MAXLENGTH times.

1.2 version 1

Second iteration of the bdamage file as magicnumber[8] = "bdam1".

Col	Field	Type	Brief description
0	MAXLENGTH	int	Number of cycles
1	ID	int	Id for mismatch type ¹
2	NREADS	int	Number of reads used supporting the mismatch matrix
3	1	float[16]	mismatch rate for first cycle from the 5prime
4	<i>i</i>	float[16]	mismatch rate for the <i>i</i> 'th cycle from the 5prime
5	MAXLENGTH	float[16]	mismatch rate for the last cycle from the 5prime
6	1	float[16]	mismatch rate for first cycle from the 3prime
7	<i>i</i>	float[16]	mismatch rate for the <i>i</i> 'th cycle from the 3prime
8	MAXLENGTH	float[16]	mismatch rate for the last cycle from the 3prime

Table 1: Content of bdamage.gz file. Note¹ This is either the taxidID or the referenceID relative to the SAM/BAM header for single species resequencing projects or it is the *taxid* if output has been generated with metadamage lca 3) Order is given by AA,AC,AG,AT,CA,CC,CG,CT,GA,GC,GG,GT,TA,TC,TG,TT, with first base indicatting reference nucleotide and second base indicating observed nucleotide

2 .lca

This section describes the test output generated by a metadamage lca subfunctionality and contains information at the readlevel regarding both taxonomic information and statistics pertaining usefull readinformation.

First line of the file begins with a hashtag followed by the actual command used for generating the file. Last entry of the line is again a hashtag followed by the git commit id which will serve as a primitive versioncontrol.

Each line consists of a number of items seperated by tabspace. First entry contains readID together with other information seperated by colon. After the first entry successive blocks of the type taxid:name:"taxlevel" from the lca toward the root. The complete specification is seen in table below.

Col	Brief description	
1	readID	readID, this might contains colon
2	seq	The actual sequence
3	length(seq)	The length of the sequence
4	nAlignments	The number of alignments used for inferring the lca
5	gc-content	The GC content for the sequence
6	lca taxid	the taxomic id (integer)
7	lca taxid	the taxonomic name(string)
8	lca "taxlevel"	the taxonomic level
9	taxid	the taxomic id (integer)
10	taxid	the taxonomic name(string)
11	"taxlevel"	the taxonomic level

Table 2: Content of a .lca file. Note that 1) seperate between fields[1-6,8-9] is tab, readID might also contain colon. 2) the quotes around field[7,8] is intentional since taxlevels and names might contain spaces andor colons. 3) Number of tab seperated entries is consistent across different reads, since separator between block 9-11 is semicolon. Seperator between 6-9,9-11 is colon

3 .stat

Very simple tabseperated flatfile

1. taxid
2. Number of supporting reads
3. Mean lengths of supporting reads
4. Variance of the lengths of the supporting reads
5. Mean gccontent of supporting reads
6. Variance of the gccontent of supporting reads
7. name of lca in quotes (if relevant, otherwise NA)
8. name of taxomic level of lca in quotes (if relevant, otherwise NA)

4 .dfit.txt.gz

The output format will be heavily dependent on the provided runmode and supplied parameters. It might contain the per file, the per reference or the per species estimate of damage.

Only the columns described in *showfits 0* remains the same across the runmode, with the *showfits 1* or *showfits 2* describes the unique columns and order which is appended to the *showfits 0* columns.

Non-boostrapping optimization:

Column name	Brief description
<i>–showfits 0</i>	
id	identifier see paragraph for details
A	Dfit statistic. Damage at position one, taking into account offset
q	per cycle decrease
c	background substitution rate or noise baseline
ϕ	Variance between beta-binomial and binomial model
llh	Likelihood for our MLE
ncall	Number of optimization calls used for obtaining our MLE
σ_D	Z value
Zfit	significance
<i>–showfits 1, i</i> signifies position inferred from <i>.bdamage.gz</i>	
<i>fwλi</i>	damage estimates forward strand
<i>fwConfλi</i>	forward strand confidence interval $\pm$
<i>bwdλi</i>	damage estimates backward strand
<i>bwdConfλi</i>	backward strand confidence interval $\pm$
<i>–showfits 2, i</i> signifies position inferred from <i>.bdamage.gz</i>	
<i>fwKi</i>	Number of deamination substitution (C $\rightarrow$ T) observations forward strand
<i>fwNi</i>	Total number of C for forward strand
<i>fwλi</i>	Damage estimates forward strand
<i>fwfi</i>	Calculated damage frequency <i>fwKi/fwNi</i>
<i>fwConfλi</i>	forward strand confidence interval $\pm$
<i>bwKi</i>	Number of deamination substitution (G $\rightarrow$ A for ds, C $\rightarrow$ T for ss) observations forward strand
<i>bwNi</i>	Total number of G for forward strand (ds) and C (ss)
<i>bwdλi</i>	Damage estimates backward strand
<i>bwfi</i>	Calculated damage frequency <i>fwKi/fwNi</i>
<i>bwdConfλi</i>	backward strand confidence interval $\pm$

Table 3: Content of a .dfit.txt.gz file. Note that entry nine to 13 is repeated for each cycle first of the 5 and then from the 3. With the total number of times repeated is given by 1.1.

Bootstrapping optimization:

Providing *-nbootstrap* > 1 numerical optimizations of the binomial distribution will also be conducted using bootstrapping methods

Column name	Brief description
<i>-showfits 0</i>	
id	identifier see paragraph for details
A	Dfit statistic. Damage at position one, taking into account offset
q	per cycle decrease
c	background substitution rate or noise baseline
ϕ	Variance between beta-binomial and binomial model
llh	Likelihood for our MLE
ncall	Number of optimization calls used for obtaining our MLE
σ_D	Z value
Zfit	significance
A_b	Dfit statistic from bootstrap estimate. Damage at position one, taking into account offset
q_b	per cycle decrease from bootstrap estimate
c_b	background substitution rate or noise baseline from bootstrap estimate
ϕ_b	Variance between beta-binomial and binomial model from bootstrap estimate
A_CI_l	Lower bound of CI for A estimate calculated from all bootstrap values
A_CI_h	Upper bound of CI for A estimate calculated from all bootstrap values
q_CI_l	Lower bound of CI for q estimate calculated from all bootstrap values
q_CI_h	Upper bound of CI for q estimate calculated from all bootstrap values
c_CI_l	Lower bound of CI for c estimate calculated from all bootstrap values
c_CI_h	Upper bound of CI for c estimate calculated from all bootstrap values
ϕ _CI_l	Lower bound of CI for ϕ estimate calculated from all bootstrap values
ϕ _CI_h	Upper bound of CI for ϕ estimate calculated from all bootstrap values
<i>-showfits 1, i signifies position inferred from .bdamage.gz</i>	
<i>fwdxi</i>	damage estimates forward strand
<i>fwdConfi</i>	forward strand confidence interval $\pm$
<i>bwdxi</i>	damage estimates backward strand
<i>bwdConfi</i>	backward strand confidence interval $\pm$
<i>-showfits 2, i signifies position inferred from .bdamage.gz</i>	
<i>fwKi</i>	Number of deamination substitution (C→T) observations forward strand
<i>fwNi</i>	Total number of C for forward strand
<i>fwdxi</i>	Damage estimates forward strand
<i>fwfi</i>	Calculated damage frequency <i>fwKi/fwNi</i>
<i>fwdConfi</i>	forward strand confidence interval $\pm$
<i>bwKi</i>	Number of deamination substitution (G→A for ds, C→T for ss) observations forward strand
<i>bwNi</i>	Total number of G for forward strand (ds) and C (ss)
<i>bwdxi</i>	Damage estimates backward strand
<i>bwfi</i>	Calculated damage frequency <i>fwKi/fwNi</i>
<i>bwdConfi</i>	backward strand confidence interval $\pm$

Table 4: Content of a .dfit.txt.gz file. Note that entry nine to 13 is repeated for each cycle first of the 5 and then from the 3. With the total number of times repeated is given by 1.1.

Depending on which parameters and runmode (local, global or lca) that was supplied to both *get-damage* and *dfit*, the content of the id will be different. If a bamfile is supplied (with *-bam*) then each line will be the information associated with the different refs in the bam file and the id will be the reference ids from the bam file. The case scenario for this would be either obtaining per chromosome estimates of damage or per reference damage which could be relevant for metagenomic studies. If user are computing the damage signal in the context of the lca. Then the id column will contain the taxid. If *-names* has been supplied to the *dfit* program, then the id column will the taxid: *scientific name*. If *-nodes* has not been defined the *dfit.txt.gz* will only contain information for the observed references.

If `-nodes` has been defined the program will aggregate the summary statistics for the internal nodes.

5 .boot.stat.txt.gz

Providing `dft` command with `nbootstrap > 1` and `doboot 1` the bootstrapping values (`id,A_b,q_b,c_b, $\phi$ _b`) for each iteration are stored in separate file

6 .agggregate.stat.txt.gz

Aggregates the information stored within the `lca .stat` format, described in section 2. The aggregated file has the prefix `.agggregate.stat.txt.gz`, with a similar format to the format presented for the `lca` output in section 2.

Col	Brief description	
1	taxid	The lca taxomic id (integer)
2	name	The lca taxonomic name(string)
3	rank	The lca taxonomic rank/level (string)
4	nalign	The aggregated number of alignments used for inferring the lca across the taxonomic levels
5	nreads	The aggregated number of reads for inferring the lca across the taxonomic levels
6	mean_rlen	The weighted mean of read length when transvering through the taxonomic levels
7	var_rlen	The pooled variance of read length when transvering through the taxonomic levels
8	mean_gc	The weighted mean of gc content when transvering through the taxonomic levels
9	var_gc	The pooled variance of gc content when transvering through the taxonomic levels
10	lca	The lca rank
11	taxa_path	The taxonimcal path from lca to root

Table 5: Content of a `.agggregate.stat.txt.gz` file. With column 10 containing `taxid` and `name` for the `lca` node separated by colon, and column 11 contains the `taxid`:"`name`" for all nodes from the `lca` to the root, with each node separated by comma, i.e. `taxid`:"`name`,`taxid`:"`name`,`taxid`:"`name`

7 .rlens.gz

Readlength distribution. Distribution is count of alignments of specific readlengths. Depending on `runmode` there might be multiple groups as `taxid/refs` and there will be group specific distributions.

7.1 Old versions (pre april 2026)

Distributions for different groups are split by newlines. First entry on each line is the identifier (`chromosomename,taxid`). The remaining entries are the number of times we have observed an alignment of length (`columnnumber`). Notice that the first 30 column of counts is likely to be zero since reads shorter than 30basepairs are normally discarded.

7.2 Newer versions: human nice format

Each distribution is a newline. Each entry in the distribution is `bin:count` seperated by tab. Example can be seen below. This is the default option that should be enforcable by `--rlens_flat_out 0`.

Output below is clipped at rightside and at bottom. Example shown is from one of the testfiles, and has been generated by `cut -c1-80 test/output/test_getdamage_local.rlens|head`

```
id rlen:count
0 35:33 36:29 37:24 38:38 39:24 40:29 41:36 42:28 43:27 44:36 45:29 46:23 47:32...
1 35:103 36:106 37:102 38:111 39:109 40:126 41:125 42:164 43:154 44:133 45:152...
2 35:35 36:44 37:28 38:41 39:33 40:36 41:41 42:33 43:34 44:43 45:41 46:36 47:40...
3 35:36 36:39 37:25 38:37 39:29 40:34 41:39 42:33 43:34 44:43 45:41 46:33 47:39...
4 35:100 36:98 37:95 38:112 39:107 40:136 41:127 42:161 43:157 44:138 45:134 46:...
5 35:92 36:118 37:110 38:125 39:129 40:168 41:158 42:164 43:195 44:180 45:177 46:...
6 35:95 36:130 37:111 38:125 39:129 40:178 41:167 42:148 43:196 44:193 45:185 46:...
7 35:138 36:145 37:131 38:125 39:147 40:179 41:190 42:205 43:196 44:201 45:220 4...
8 35:95 36:132 37:113 38:137 39:133 40:182 41:162 42:176 43:196 44:190 45:189 46...
...
```

7.3 Newer versions: human nice format

The parsing of the standard output rlens file can be abit confusing for programs to parse due to the varying columnwidth. Therefore by supplying `--rlens_flat.out 1` or `-z 1` in relevant commands the output will be flattened in to a simpler but longer format.

```
IFILE=./data/f570b1db7c.dedup.filtered.rname.bam
../metaDMG-cpp getdamage -r 1 -l 35 -p 5 -o output/test_getdamage_local ${IFILE} -z 1
gunzip -c output/test_getdamage_local.rlens.gz|head -n20
##below is the output from the last command above
ID rlen count
0 35 33
0 36 29
0 37 24
0 38 38
0 39 24
0 40 29
0 41 36
0 42 28
0 43 27
0 44 36
0 45 29
0 46 23
0 47 32
0 48 23
0 49 25
0 50 17
0 51 16
0 52 19
0 53 17
```